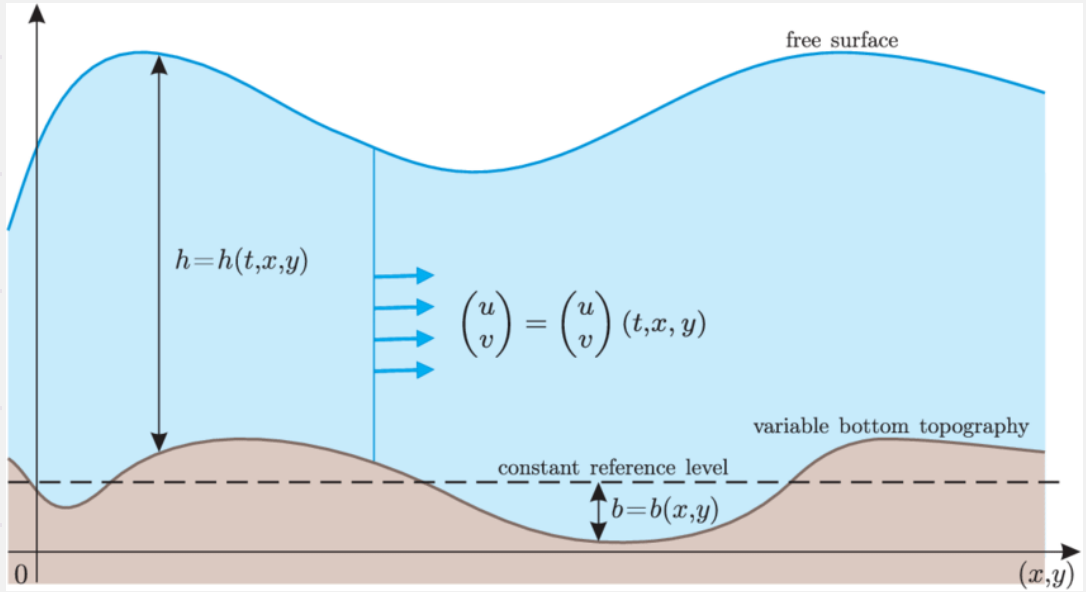
High Performance Scientific Computing

Students:
Tommaso Botarelli s2404989
Francesco Moglia s2405535

Presentation outline

- Problem description
- Serial implementation
  - Bilinear interpolation
  - Stability analysis
  - Aritmethic intensity analysis
  - Serial bottlenecks
- Going parallel
  - MPI
  - OpenMP
  - Hybrid approach MPI + OpenMP
- Results discussion
- GPU parallelization
- Real bathymetry data
- Future developments

Physical problem: shallow water model



$$\frac{\partial \eta}{\partial t} = -\nabla \cdot (h\mathbf{u})$$

$$\frac{\partial \mathbf{u}}{\partial t} = -g\nabla\eta - \gamma\mathbf{u}$$

where the unknown fields $\eta(t,x,y)$ and $\mathbf{u}(t,x,y) = (u(t,x,y), v(t,x,y), 0)$ are respectively the free-surface elevation and the depth-averaged velocity, and where

- $h(x,y)$ is the depth at rest;
- g is the gravitational acceleration;
- γ is the dissipation coefficient.

Bilinear interpolation

Bilinear interpolation is used to enhance the accuracy of converting bathymetry data into datapoints. This substitutes the original nearest neighbor algorithm

```
#define GET_X_COORD(data, i) ((data)->dx * (i))
#define GET_Y_COORD(data, i) ((data)->dy * (i))
```

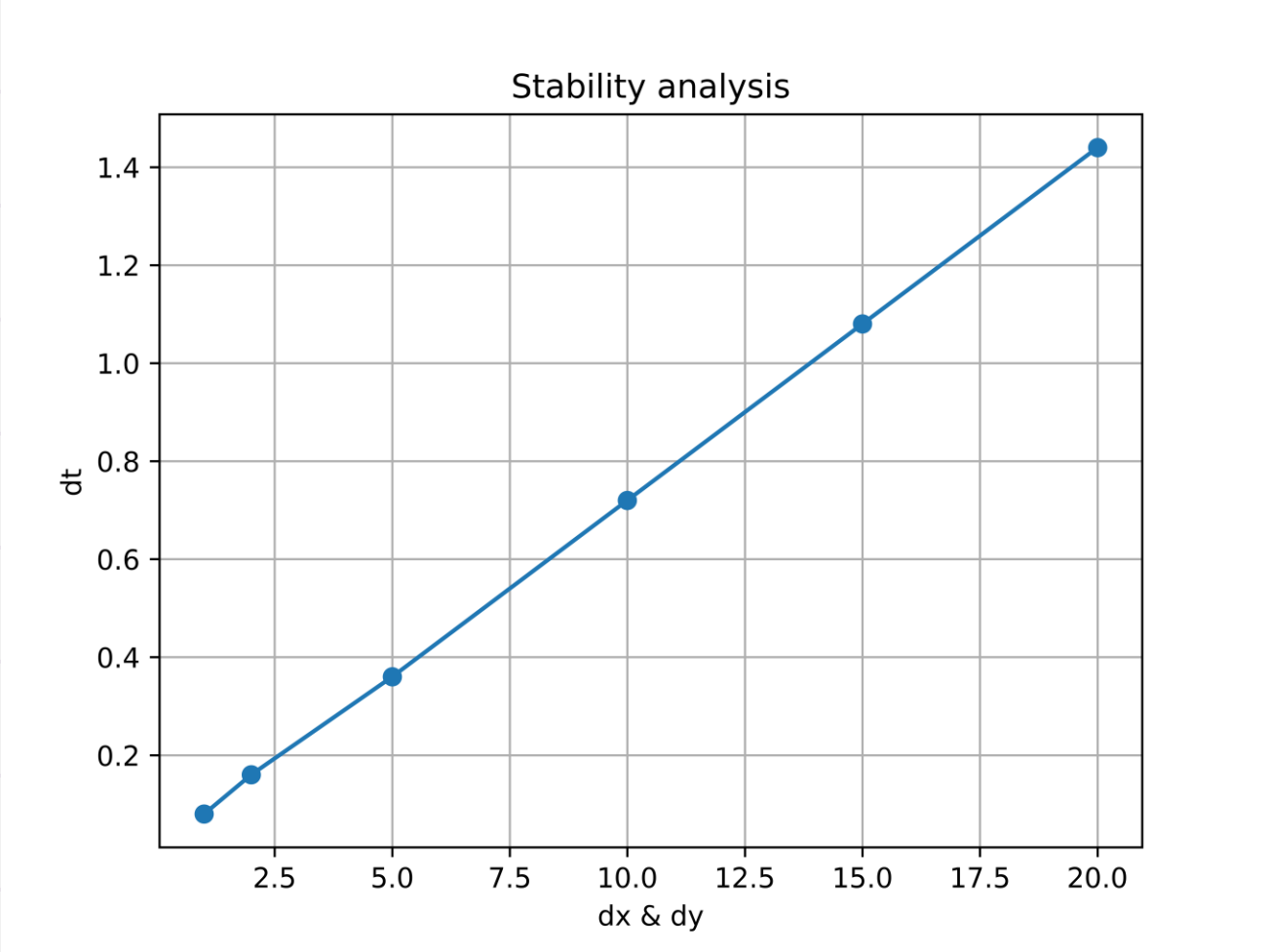
```
double interpolate_data(const struct data *data, double x, double y)
{
  double real_i = x / data->dx;
  double real_j = y / data->dy;

  int i = floor(real_i);
  int j = floor(real_j);

  if(i < 0) i = 0;
  else if(i > data->nx - 1) i = data->nx - 1;
  if(j < 0) j = 0;
  else if(j > data->ny - 1) j = data->ny - 1;
```

```
    val = (GET(data, i, j) * (GET_X_COORD(data, i+1) - x) * (GET_Y_COORD(data, j+1) - y)
                + GET(data, i+1, j) * (x - GET_X_COORD(data, i)) * (GET_Y_COORD(data, j+1) - y)
                + GET(data, i, j+1) * (GET_X_COORD(data, i+1) - x) * (y - GET_Y_COORD(data, j))
                + GET(data, i+1, j+1) * (x - GET_X_COORD(data, i)) * (y - GET_Y_COORD(data, j))) / (data->dx * data->dy);
  }
```

The interpolation is done between the four closest point of the bathymetry

Stability analysis

- **Stability analysis ensures numerical methods produce reliable and physically meaningful results** across varying discretizations
- We evaluated the code using fixed spatial discretization steps (*dx* and *dy*) while varying the temporal discretization step (*dt*) to determine the conditions under which the solution becomes unstable (NaN or very big/small values different from expected)
- **Linear relation between space and time**



Arithmetic intensity analysis

If ***n*** is the number of data items that an algorithm operates on, and ***f(n)*** the number of operations it takes, then the arithmetic intensity is ***f(n)/n***

```
for(int n = 0; n < nt; n++) {
```

```
// impose boundary conditions
double t = n * param.dt;
if(param.source_type == 1) {
  // sinusoidal velocity on top b
  double A = 5;
  double f = 1. / 20.;
  for(int i = 0; i < nx; i++) {
    for(int j = 0; j < ny; j++) {
      SET(&u, 0, j, 0.);
      SET(&u, nx, j, 0.);
      SET(&v, i, 0, 0.);
      SET(&v, i, ny, A * sin(2 *
    }
  }
}
```

```
// update eta
for(int i = 0; i < nx; i++) {
  for(int j = 0; j < ny ; j++) {

    double h_x_u_i1_j = GET(&h_interp_u, i+1,
    double h_x_u_i_j = GET(&h_interp_u, i, j)

    double h_x_v_i_j1 = GET(&h_interp_v, i, j
    double h_x_v_i_j = GET(&h_interp_v, i, j)

    double eta_ij = GET(&eta, i, j)
                - param.dt / param.dx * (h_x_u_i1
                - param.dt / param.dy * (h_x_v_i_

    SET(&eta, i, j, eta_ij);
  }
}
```

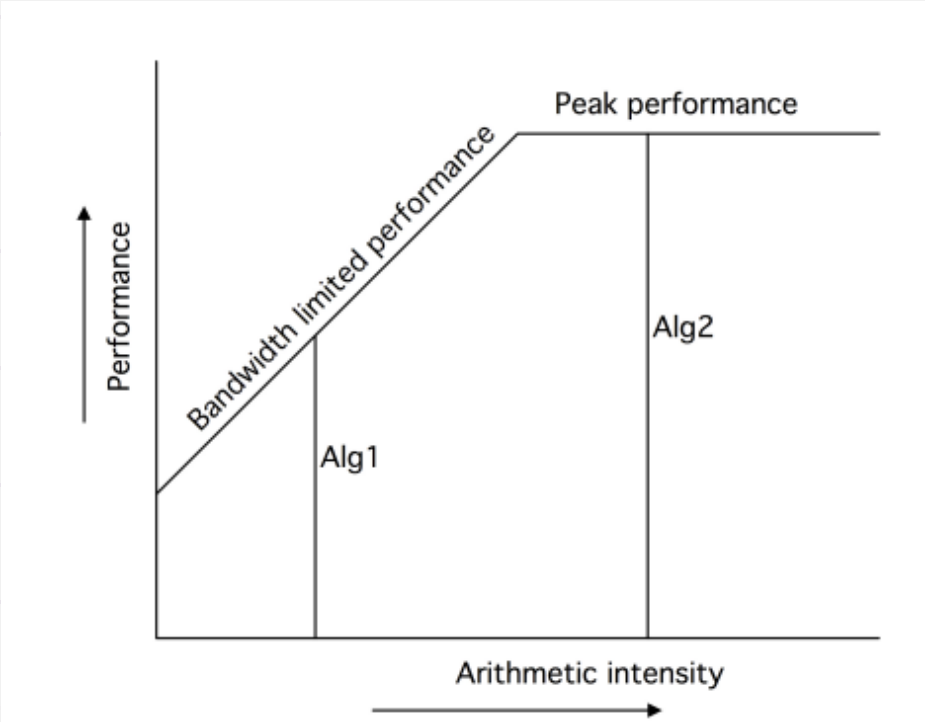
```
// update u and v
for(int i = 0; i < nx; i++) {
  for(int j = 0; j < ny; j++) {
    double c1 = param.dt * param.g;
    double c2 = param.dt * param.gamma;
    double eta_ij = GET(&eta, i, j);
    double eta_imj = GET(&eta, (i == 0) ? 0 : i - 1, j);
    double eta_ijm = GET(&eta, i, (j == 0) ? 0 : j - 1);
    double u_ij = (1. - c2) * GET(&u, i, j)
      - c1 / param.dx * (eta_ij - eta_imj);
    double v_ij = (1. - c2) * GET(&v, i, j)
      - c1 / param.dy * (eta_ij - eta_ijm);
    SET(&u, i, j, u_ij);
    SET(&v, i, j, v_ij);
  }
}
```

Arithmetic intensity analysis

If ***n*** is the number of data items that an algorithm operates on, and ***f(n)*** the number of operations it takes, then the arithmetic intensity is ***f(n)/n***

f(n)	= 27 Flops * (800 * 800) elements	= 17 280 000 Flops / cycle
n	= 3 data structure x (800 * 800) elements * 8 bytes / element	= 15 360 000 bytes / cycles

Arithmetic intensity = 1.125 Flops / bytes



NIC5 single core theoretical performance:

- Bandwidth = 25.6 GB/s
- Processing Power = 46.4 GFLOP/s

Roofline model peak performance :
min(Processing Power, Bandwidth x Arithm. Int.)
min(46.4 , 28.8) GFLOP/s

➔ **We are memory bound**

Arithmetic intensity analysis

What if we could use the capabilities of one entire NIC5 processor?

AMD EPYC 7542 processor:

Bandwidth = 204.8 GB/s
Processing Power = 1.485 TFLOP/s

Peak performance = min(Processing Power, Bandwidth x Arithm. Int.)

= min(1485 , 230.4) GFLOP/s = 230.4 GFLOP/s

➔ Performance scales only with bandwidth and so we can't make use of the full capabilities

A processor with 200 GB/s of memory bandwidth and 2.8 TFLOPS/s of processing power would obtain a similar performance even if its processing power is double of what we have on NIC5 processors.

Serial bottlenecks

Multiple bottlenecks were found in the initial serial implementation:

1. **Useless nested cycles for boundary conditions setting**
2. Nested loops not optimized for *eta* update
3. Nested loops not optimized for *u* and *v* update

```
// sinusoidal velocity on top boundary
double A = 5;
double f = 1. / 20.;
for(int i = 0; i < nx; i++) {
    for(int j = 0; j < ny; j++) {
        SET(&u, 0, j, 0.);
        SET(&u, nx, j, 0.);
        SET(&v, i, 0, 0.);
        SET(&v, i, ny, A * sin(2 * M_PI * f * t));
    }
}
```

```
// sinusoidal velocity on top boundary
double A = 5;
double f = 1. / 20.;
double a_sin_value = A * sin(2 * M_PI * f * t);
for(int j = 0; j < ny; j++) {
    SET(&u, 0, j, 0.);
    SET(&u, nx, j, 0.);
}
for(int i = 0; i < nx; i++) {
    SET(&v, i, 0, 0.);
    SET(&v, i, ny, a_sin_value);
}
```

The boundary conditions setting doesn't need the nested cycle.
Divide it into two separate cycles allow better performance

Serial bottlenecks

Multiple bottlenecks were found in the initial serial implementation:

1. Useless nested cycles for boundary conditions setting
2. **Nested loops not optimized for *eta* update**
3. Nested loops not optimized for *u* and *v* update

```
for(int i = 0; i < nx; i++) {
    for(int j = 0; j < ny ; j++) {
        double h_x_u_i1_j = GET(&h_interp_u, i+1, j);
        double h_x_u_i_j = GET(&h_interp_u, i, j);

        double h_x_v_i_j1 = GET(&h_interp_v, i, j+1);
        double h_x_v_i_j = GET(&h_interp_v, i, j);

        double eta_ij = GET(&eta, i, j)
                - param.dt / param.dx * (h_x_u_i1_j * GET(&u, i+1, j) - h_x_u_i_j * GET(&u, i, j))
                - param.dt / param.dy * (h_x_v_i_j1 * GET(&v, i, j+1) - h_x_v_i_j * GET(&v, i, j));

        SET(&eta, i, j, eta_ij);
    }
}
```

```
for(int j = 0; j < ny ; ++j) {
    for(int i = 0; i < nx; ++i) {
        double h_x_u_i1_j = GET(&h_interp_u, i+1, j);
        double h_x_u_i_j = GET(&h_interp_u, i, j);

        double h_x_v_i_j1 = GET(&h_interp_v, i, j+1);
        double h_x_v_i_j = GET(&h_interp_v, i, j);

        double eta_ij = GET(&eta, i, j)
                - dt_dx * (h_x_u_i1_j * GET(&u, i+1, j) - h_x_u_i_j * GET(&u, i, j))
                - dt_dy * (h_x_v_i_j1 * GET(&v, i, j+1) - h_x_v_i_j * GET(&v, i, j));

        SET(&eta, i, j, eta_ij);
    }
}
```

- The base implementation is not optimized for data access.
- The order of the nested cycles was changed allowing **spatial locality**
  - Higher probability of using a data already in the registers → lower number of cache requests
  - Higher cache hit probability
- The constant multiplications are done outside the main loop allowing a smaller number of operations.

Serial bottlenecks

Multiple bottlenecks were found in the initial serial implementation:

1. Useless nested cycles for boundary conditions setting
2. Nested loops not optimized for *eta* update
3. **Nested loops not optimized for *u* and *v* update**

```
for(int i = 0; i < nx; i++) {
    for(int j = 0; j < ny; j++) {
        double c1 = param.dt * param.g;
        double c2 = param.dt * param.gamma;
        double eta_ij = GET(&eta, i, j);
        double eta_imj = GET(&eta, (i == 0) ? 0 : i - 1, j);
        double eta_ijm = GET(&eta, i, (j == 0) ? 0 : j - 1);
        double u_ij = (1. - c2) * GET(&u, i, j)
            - c1 / param.dx * (eta_ij - eta_imj);
        double v_ij = (1. - c2) * GET(&v, i, j)
            - c1 / param.dy * (eta_ij - eta_ijm);
        SET(&u, i, j, u_ij);
        SET(&v, i, j, v_ij);
    }
}
```

```
for(int j = 0; j < ny; j++) {
    for(int i = 0; i < nx; i++) {
        double eta_ij = GET(&eta, i, j);
        double eta_imj = GET(&eta, (i == 0) ? 0 : i - 1, j);
        double eta_ijm = GET(&eta, i, (j == 0) ? 0 : j - 1);
        double u_ij = one_minus_c2 * GET(&u, i, j)
            - c1_dx * (eta_ij - eta_imj);
        double v_ij = one_minus_c2 * GET(&v, i, j)
            - c1_dy * (eta_ij - eta_ijm);
        SET(&u, i, j, u_ij);
        SET(&v, i, j, v_ij);
    }
}
```

Similarly to the previous optimization the two cycles are reordered allowing spatial locality and a smaller number of operations

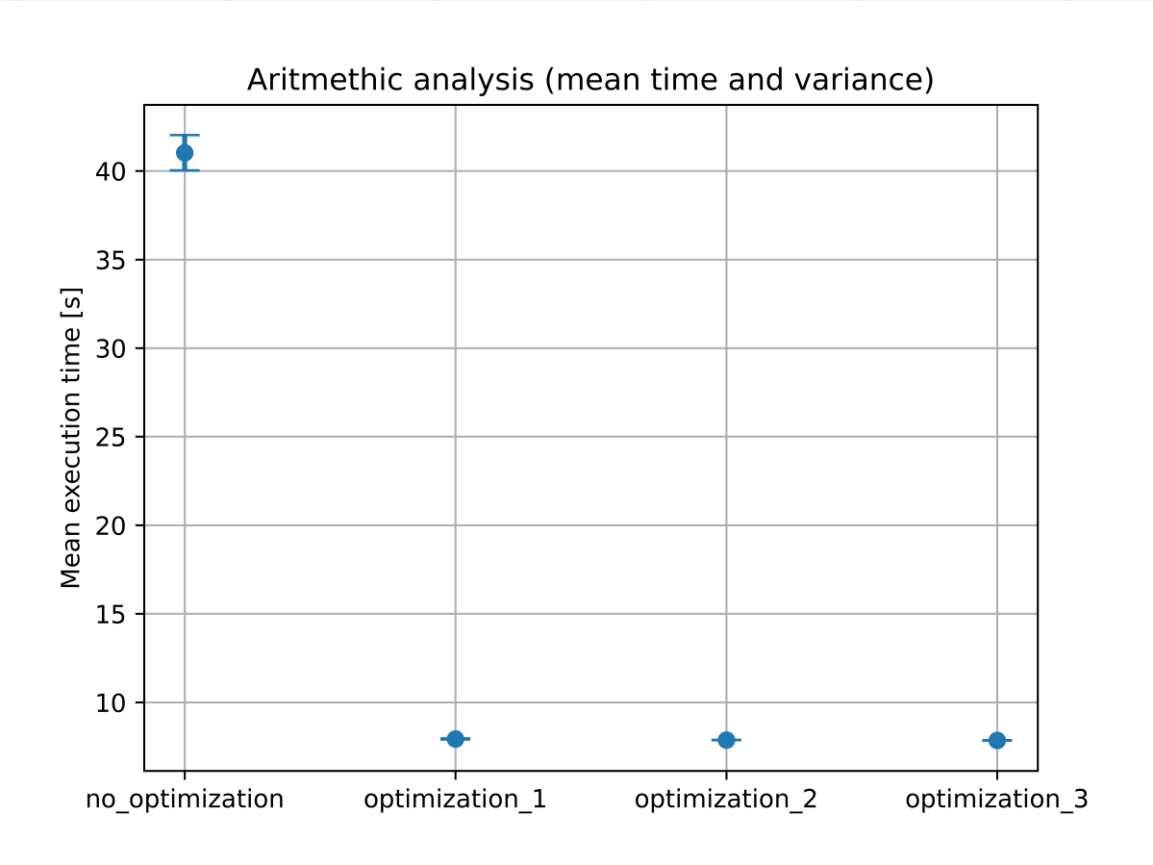
Arithmetic analysis

Three different optimizations were carried out:

1. Nested cycles reordering
2. 1 + constant calculation outside the loop but inside the main loop
3. 2 + constant calculation outside the main loop

The average execution times of 50 executions for each optimization shows that:

- The best performance improvements are reached optimizing the data access (*no_optimization -> optimization_1)*
- The following optimizations doesn't improve the performances with the same intensity
- The variance of *no_optimization is due to a* more *"random"* probability in the number of data cache hits



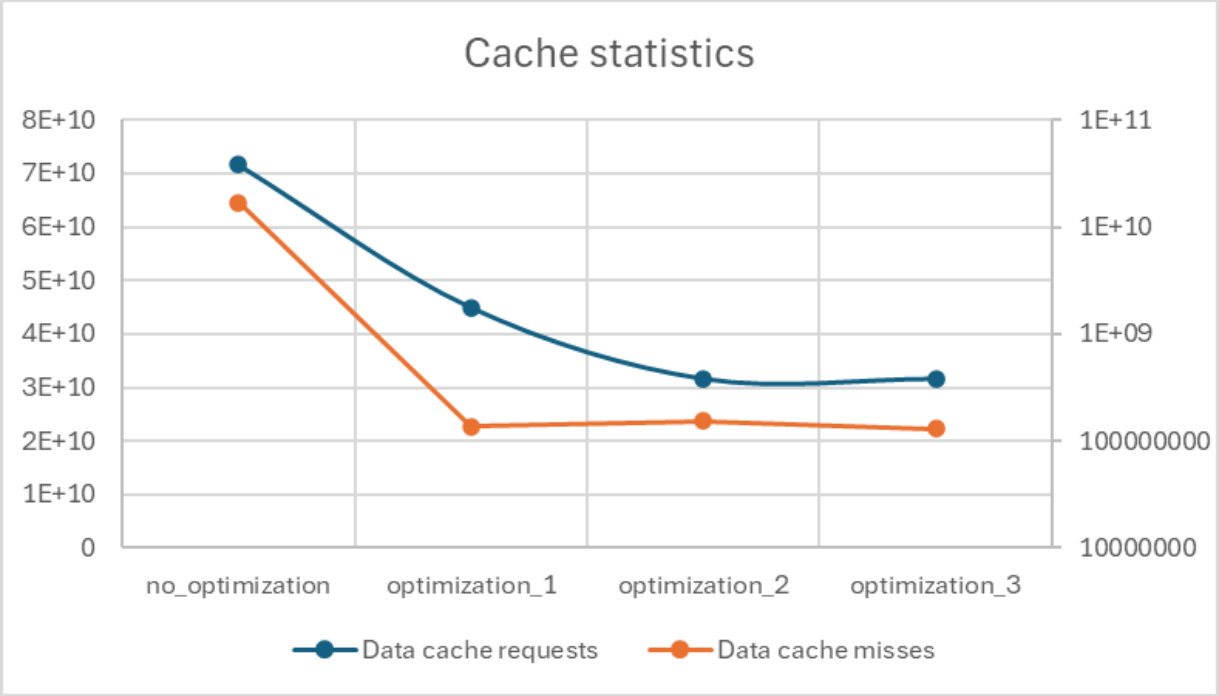
Mean execution times for standard parameters settings:
dx=5, dy=5, dt=0.05, max_time=200
Values: `[41.032, 7.929, 7.865, 7.848]`

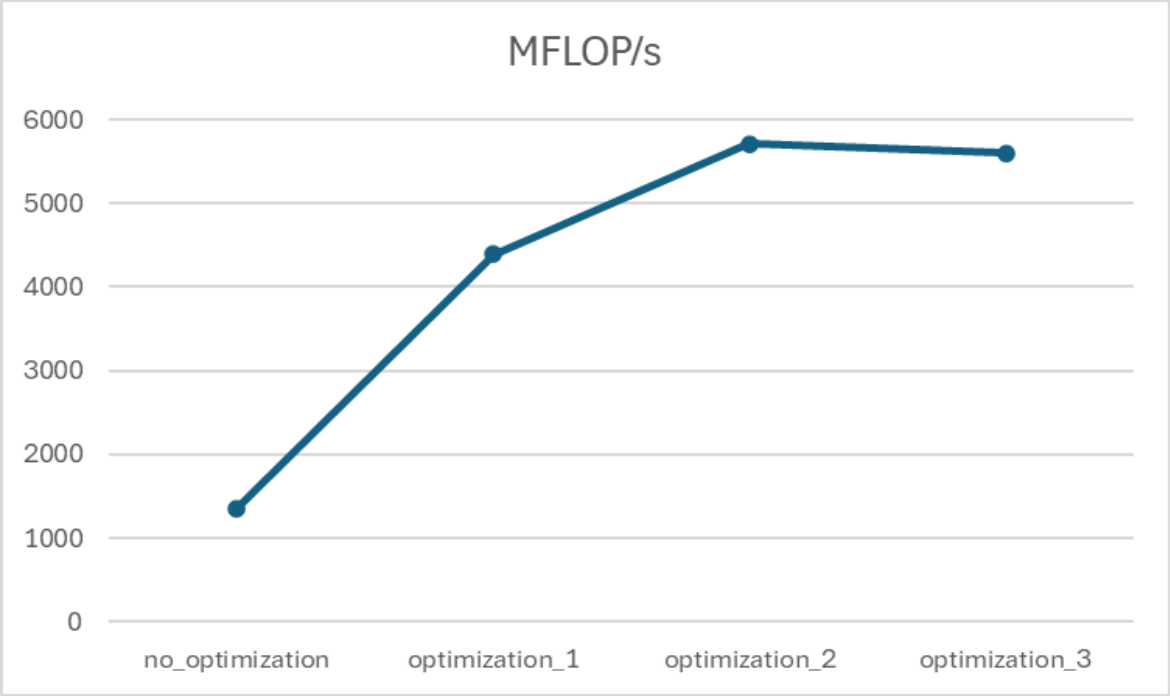
Arithmetic analysis

Likwid was used to collect data on MFLOP/s and data cache requests/misses.

The data validates the assumptions:

- Big reduction of data cache misses between *no_optimization* and *optimization_1* due to optimized data access
- Linear reduction of the number of data cache requests and bigger MFLOP/s due to optimized location of constant calculations

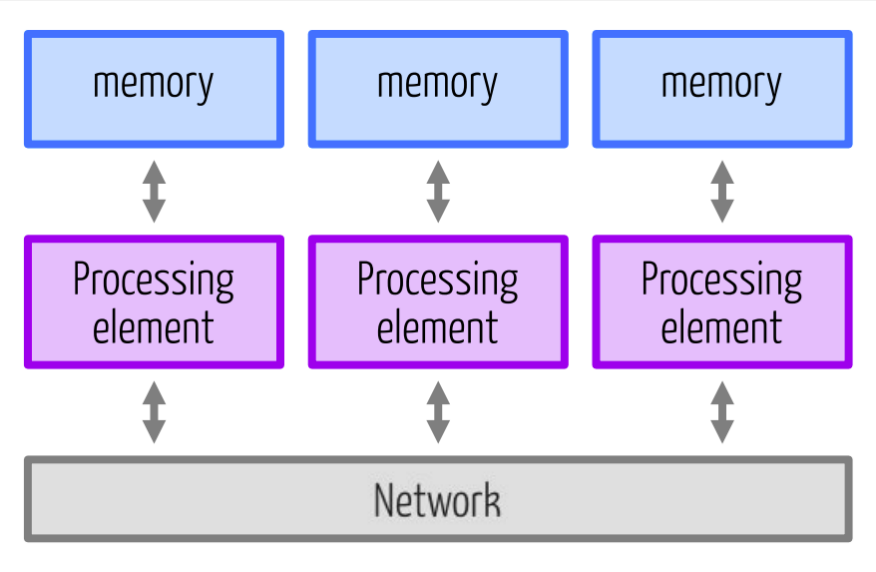




Data obtained from a single run with standard parameters settings:
dx=5, dy=5, dt=0.05, max_time=200

MPI Implementation

To make use of the multi core architetture of NIC5 and decrease the execution time of the code we can divide the domain into sub-domains and assign each one of them to a different processing elements.



Because processing elements operate as a distributed memory system, they:

- have their own private memory
- cannot have a direct access to the memory of the other processing elements

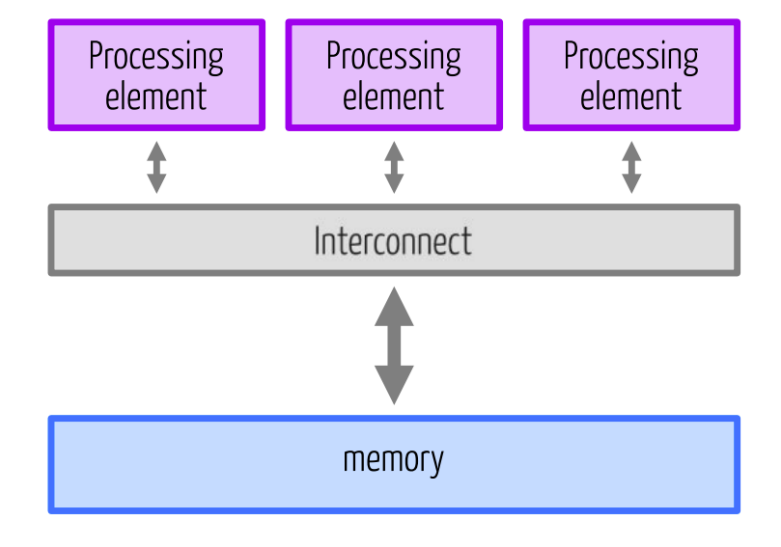
Thus, additional communication is necessary to update the boundary of each sub-domain.

MPI is an interface that provides specific functionalities to manage this inter-process communication.

In particular we implemented a **CARTESIAN TOPOLOGY** with **NON-BLOCKING COMMUNICATIONS**

OpenMP Implementation

To further divide the load of the algorithm we can split the work of each processing element and assigning it to threads.



Because threads operate as a shared memory system, they:

- have access to the entire memory of the system via an interconnect
- can read and write the memory

Nevertheless, threads have to be managed to ensure proper synchronization and work distribution

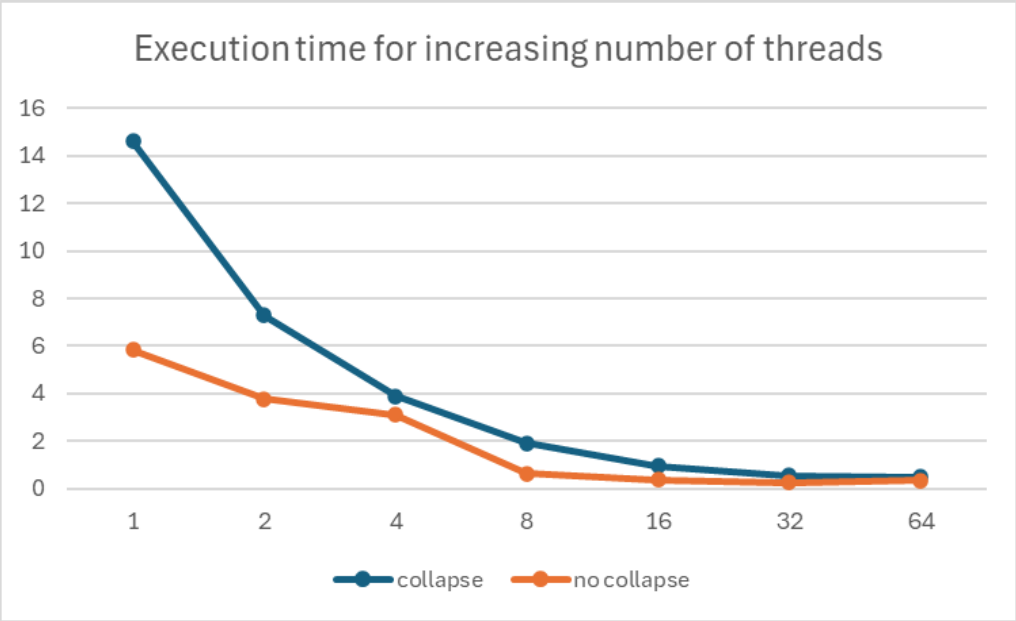
OpenMP is an interface that provides directives to parallelize the code avoiding the need for low-level details
These directives were used to define the parallel section of our code that consists in the two for loops

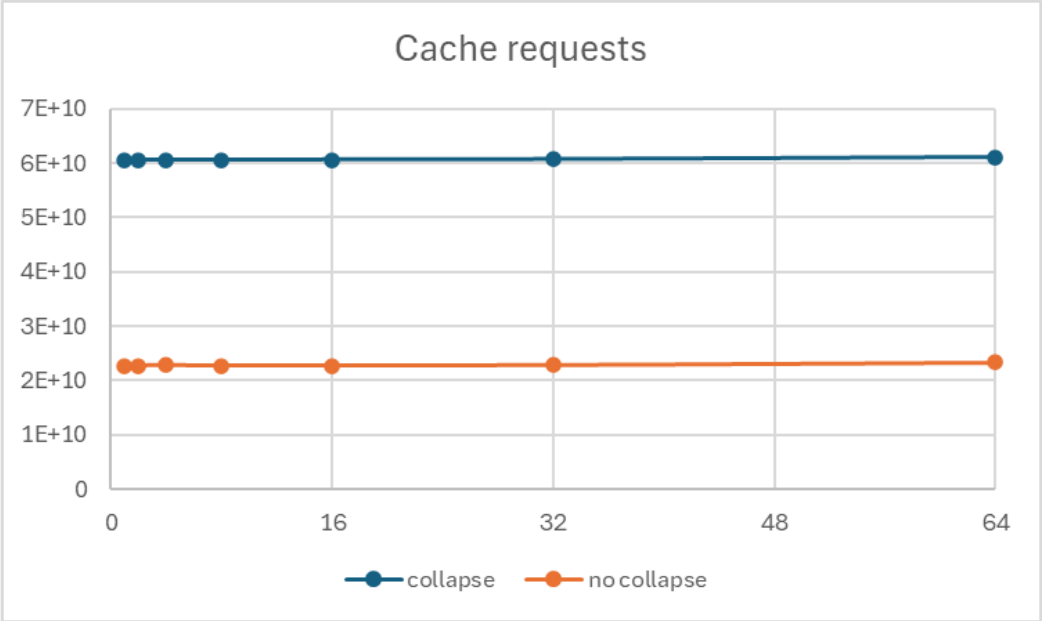
```
#pragma omp parallel for collapse(2)
for(int j = 0; j < ny ; ++j) {
  for(int i = 0; i < nx; ++i) {
```

OpenMP the collapse clause

Particular attention was given to the **collapse** clause.
This can be used to flatten two nested cycles allowing to create bigger parallel sections.

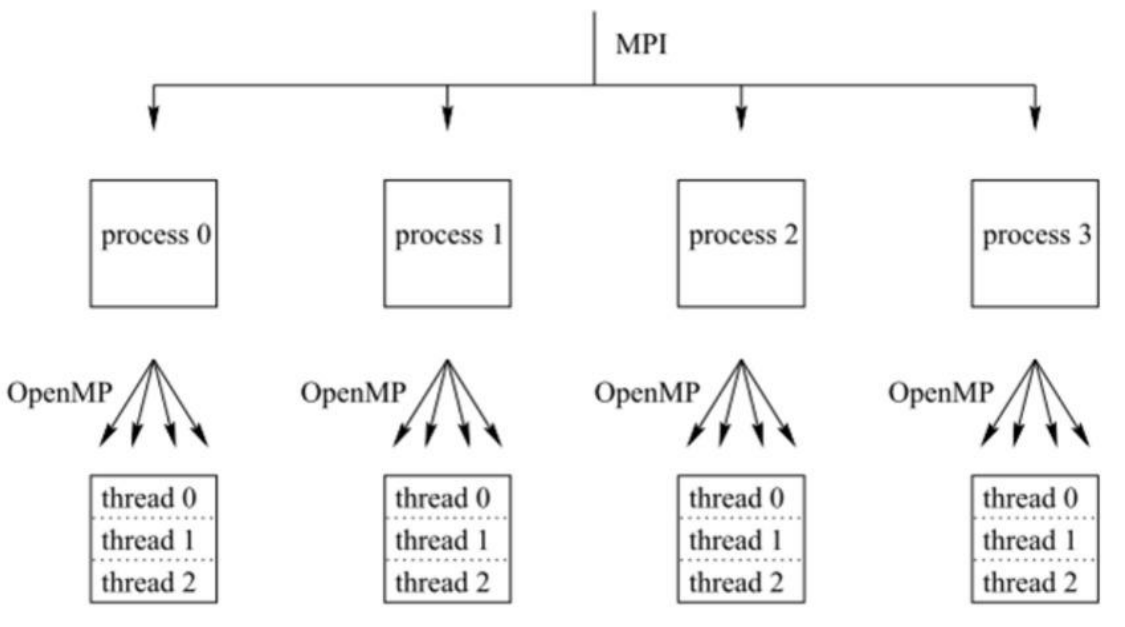
In doing so we lose some of the cache optimization that characterized the loops and thus this does not improve the performance of our application.





Data obtained from a single run wth parameters settings:
dx=5, dy=5, dt=0.035, max_time=100

MPI + OpenMP implementation



- Combining the two techiniques allows to reach the maximum performance in clusters especially for scientific simulation.
- The hybrid approach allows to reduce the execution time and/or to solve problems with bigger domain (both spatial and temporale).
- The code was built by **merging** the two different implementation (OpenMP and MPI) in one.

Scalability analysis

To study of the performance of these method both strong and weak scaling analysis were performed

Strong scaling consists in fixing the problem complexity while increasing the allocated resources.
In weak scaling on the other hand the complexity scales together with the resources.

dt was selected to maintain a stable simulation.
The max time of the simulation was decreased to reduce the execution time.

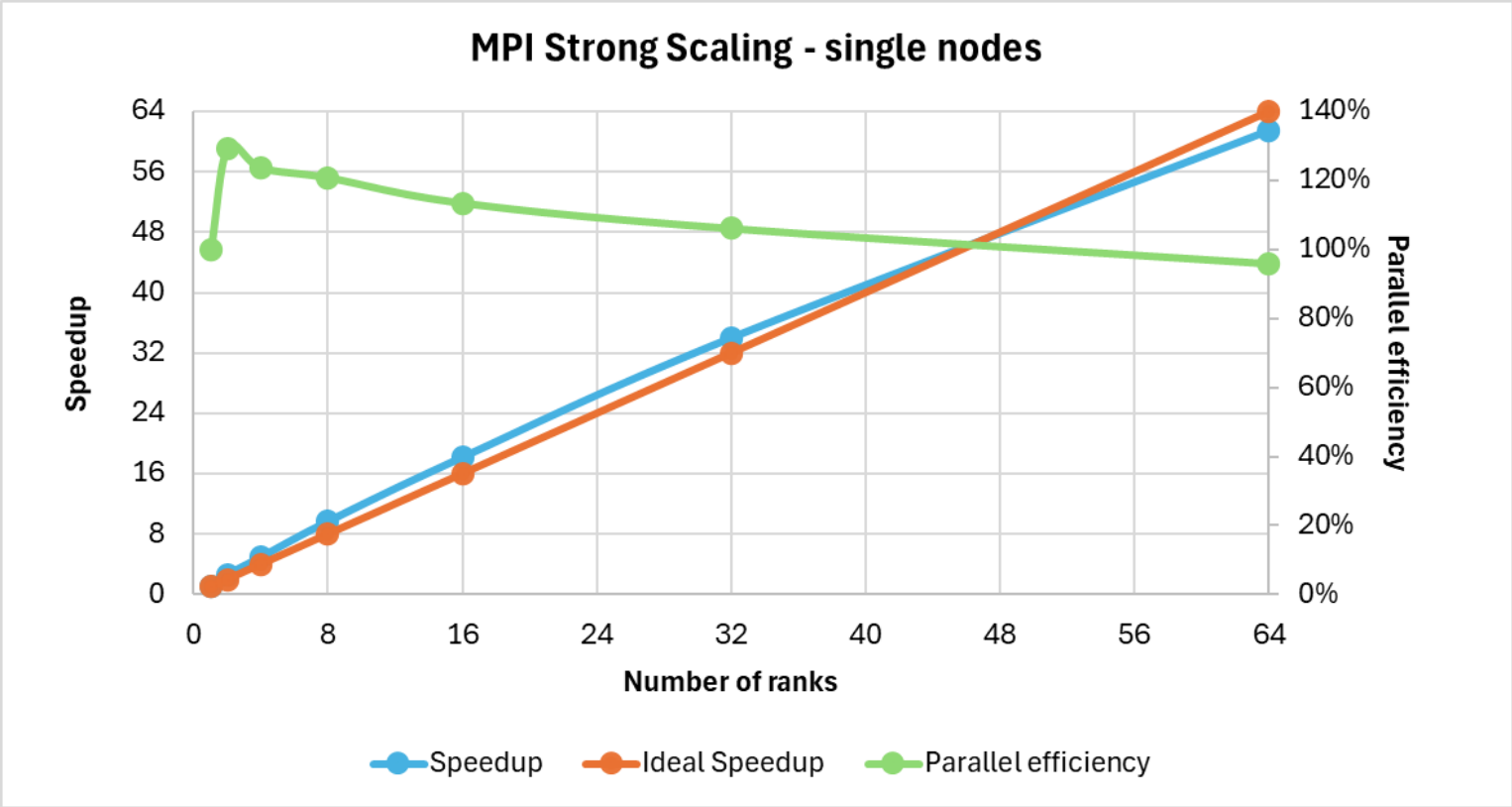
dx	dy	dt	max time
5	5	0.035	100

Strong scaling parameters

# ranks threads	dx	dy	# domain grid points
1	5	5	640000
2	5	2.5	1280000
4	2.5	2.5	2560000
8	2.5	1.25	5120000
16	1.25	1.25	10240000
32	1.25	0.625	20480000
64	0.625	0.625	40960000

Weak scaling parameters

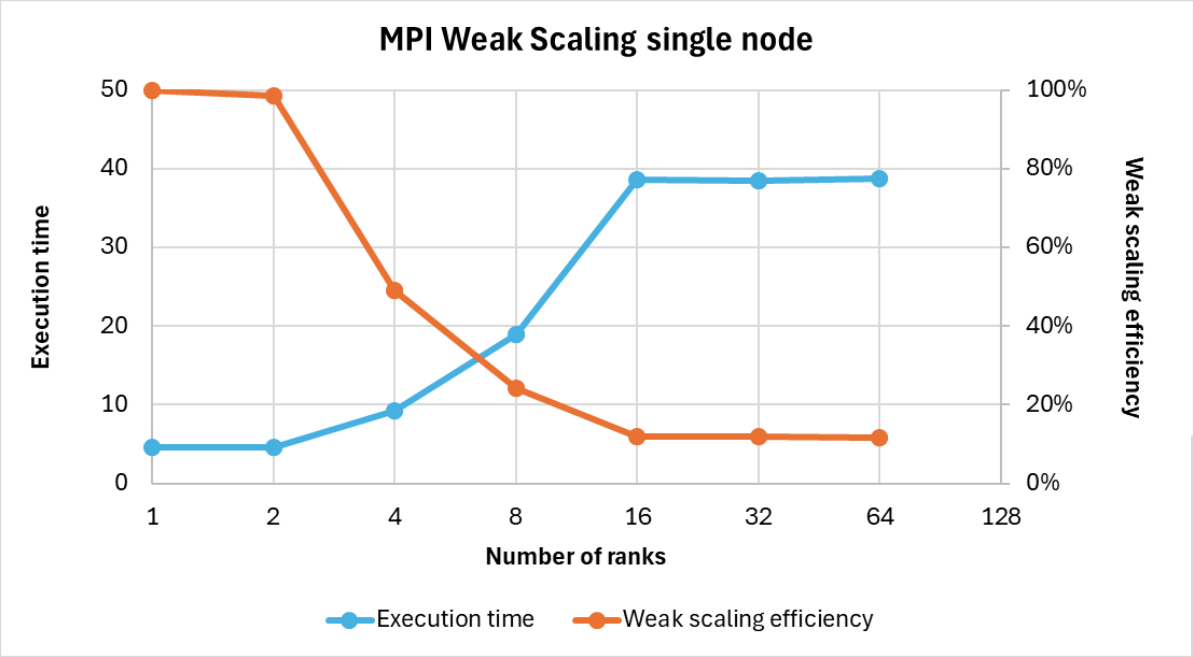
Scalability analysis: MPI strong scaling



Initial Efficiency Increase:
The initial superlinear speedup (efficiency > 100%) is likely due to improved cache utilization.

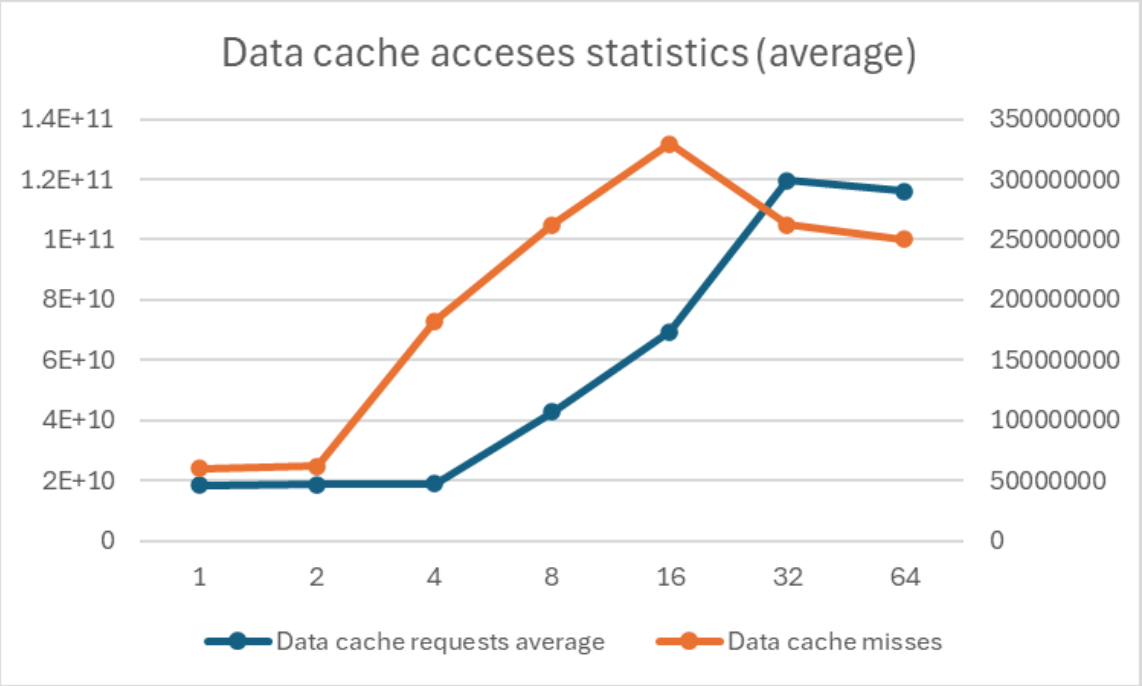
Efficiency Decline:
As the number of ranks increases, the communication overhead and the relative weight of inter-process messaging grow. This increase in communication costs progressively reduces efficiency.

Scalability analysis: MPI weak scaling (close)

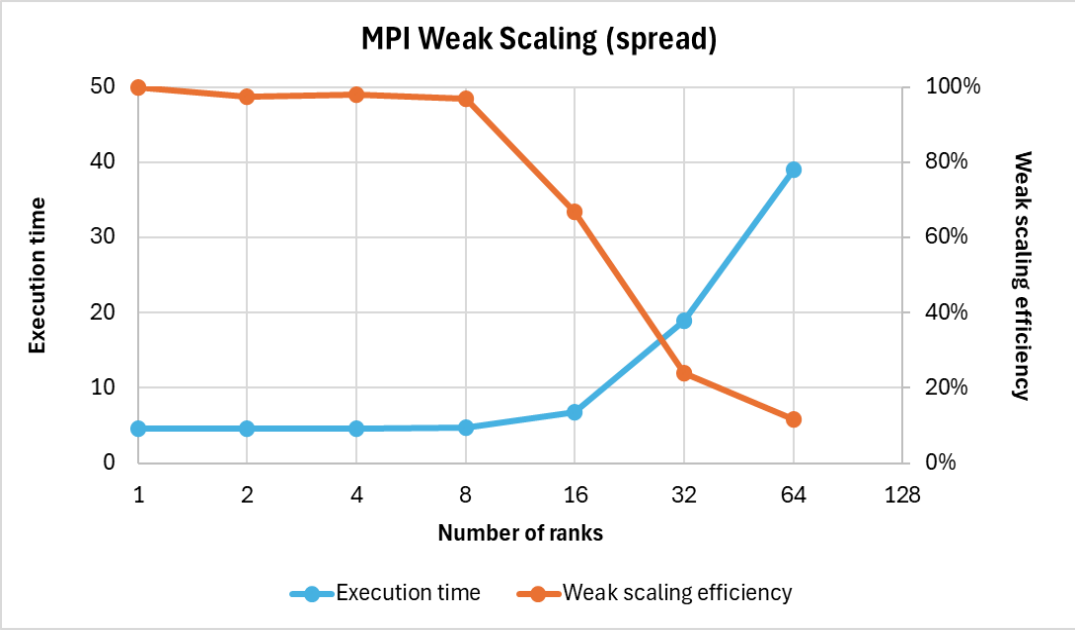


The efficiency starts diminishing from 2 ranks

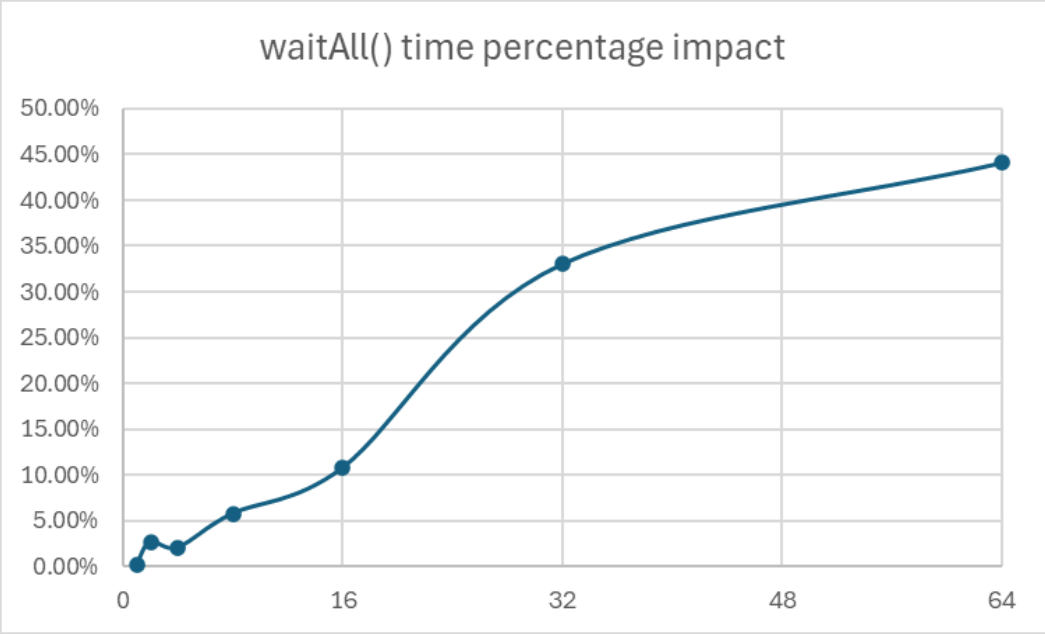
The same trend was noticed in data cache requests and misses.
In this case **cache performance is the main contributor to efficiency decline**



Scalability analysis: MPI weak scaling (spread)

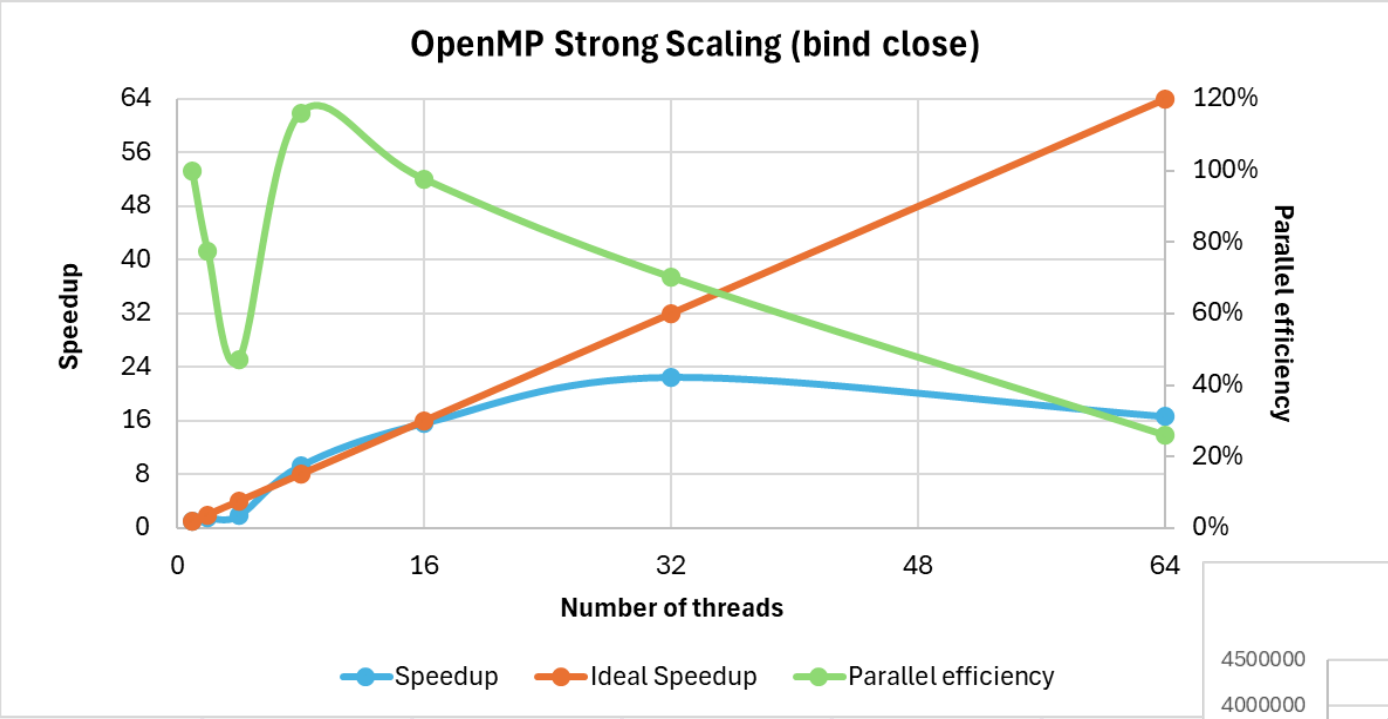


Efficiency decline
Using more than 8 ranks leads to bigger time impact of *waitAll()* function on the overall execution time.
This is likely to be caused by increased bandwidth contention between cores.

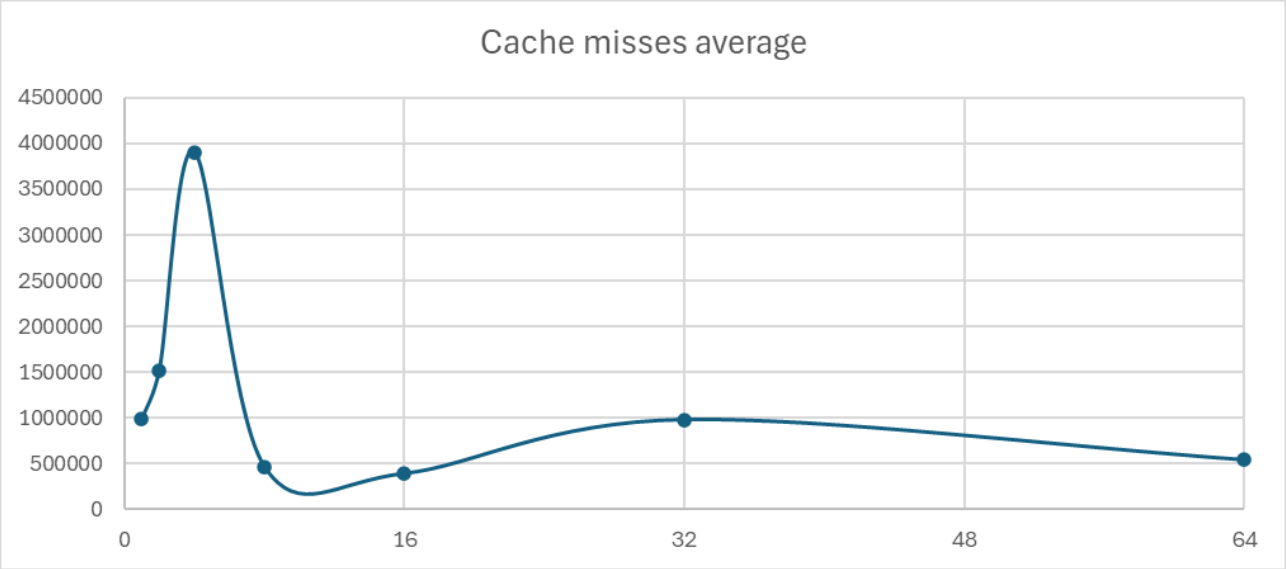


Indeed after 8 ranks the processes are distributed in multiple NUMA cores (due to spread clause).
This cause **bandwidth contention** and unbalanced waiting between cores.

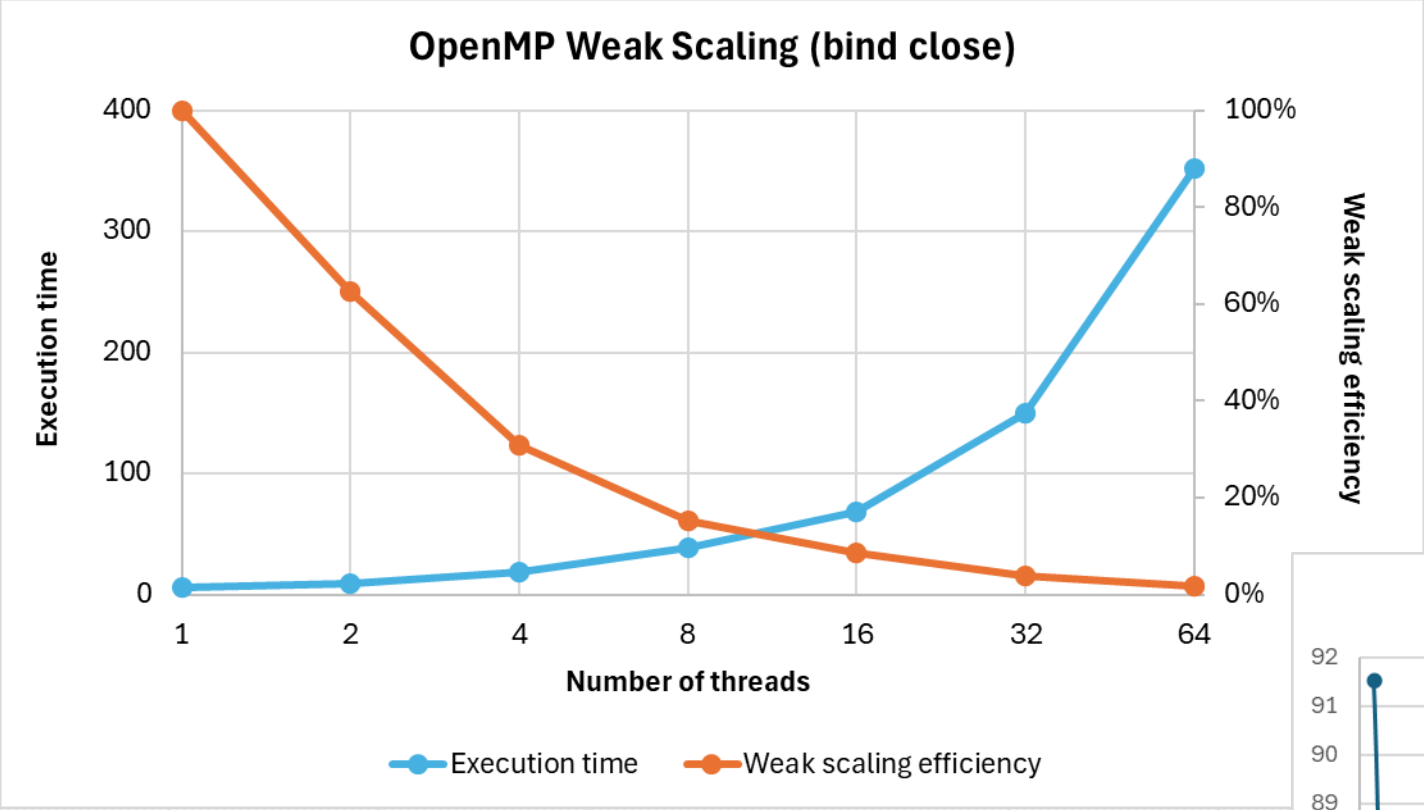
Scalability analysis: OpenMP strong scaling



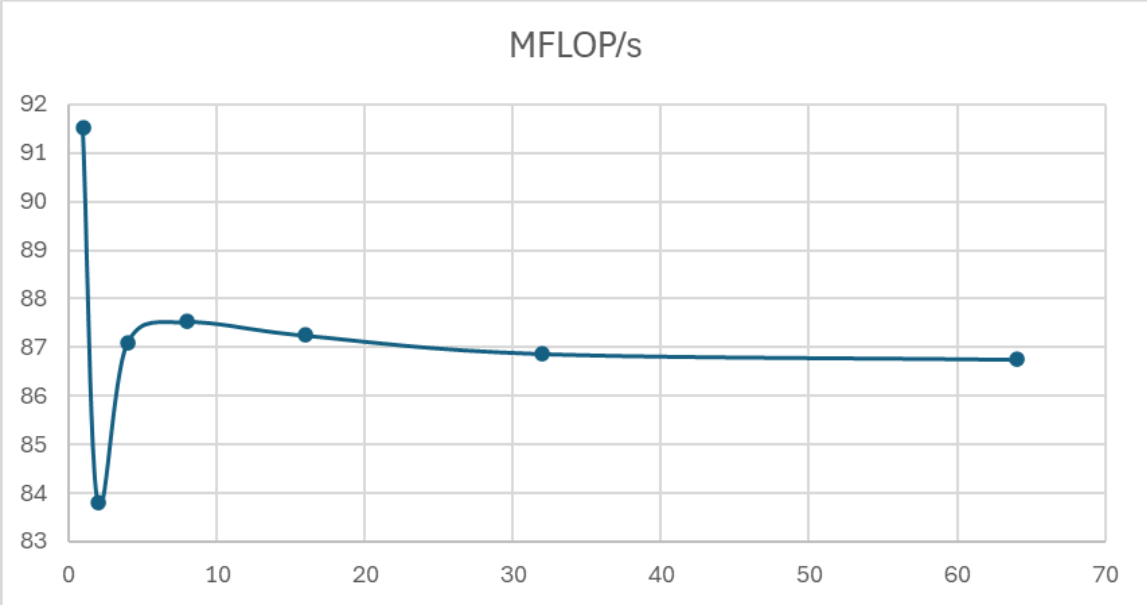
In the strong scaling of OpenMP we see again how **cache performance is the main contributor to efficiency**.



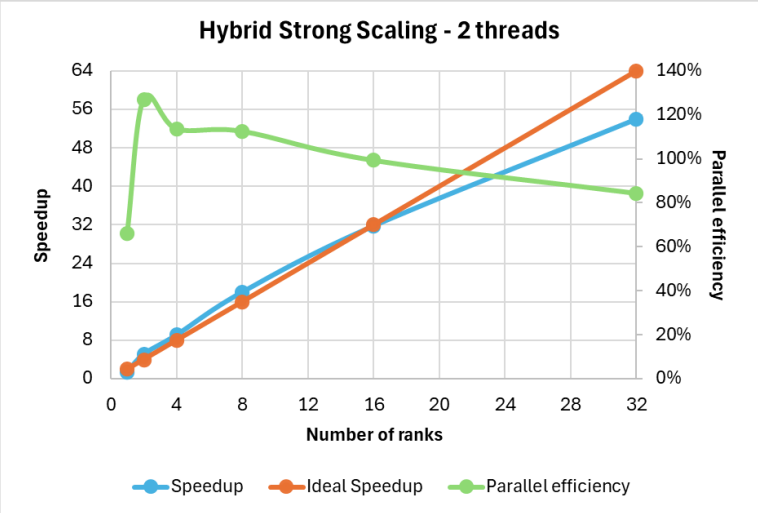
Scalability analysis: OpenMP weak scaling

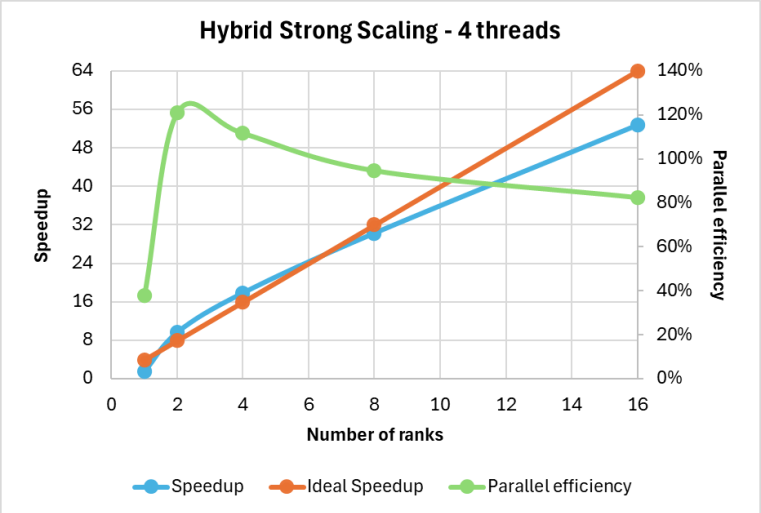


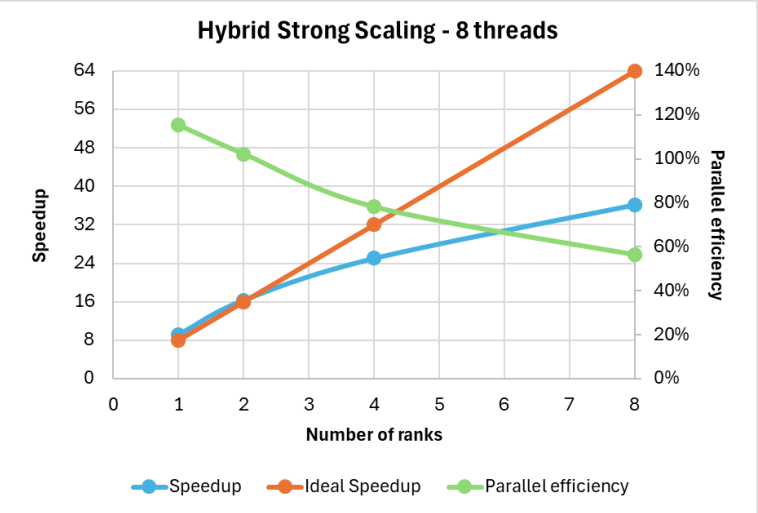
NO speedup:
The weak scaling of OpenMP gives no speedup.
This is explained by the memory boundness of the algorithm: being the bandwidth the limiting factor for performance **threading is of no use**



Scalability analysis: Hybrid strong scaling



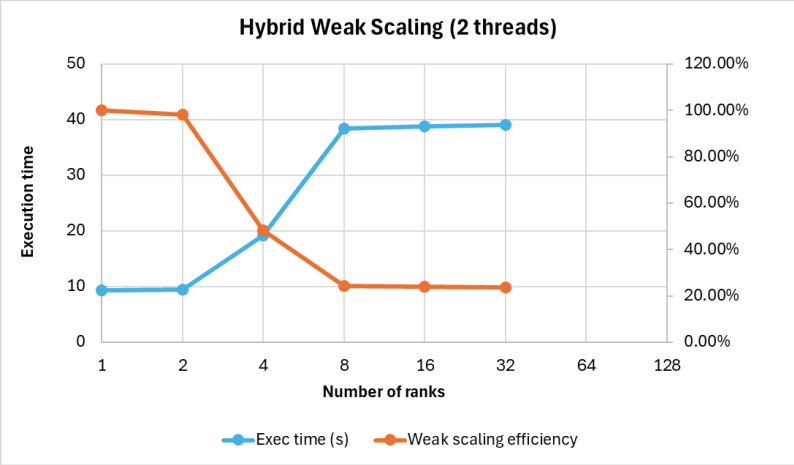


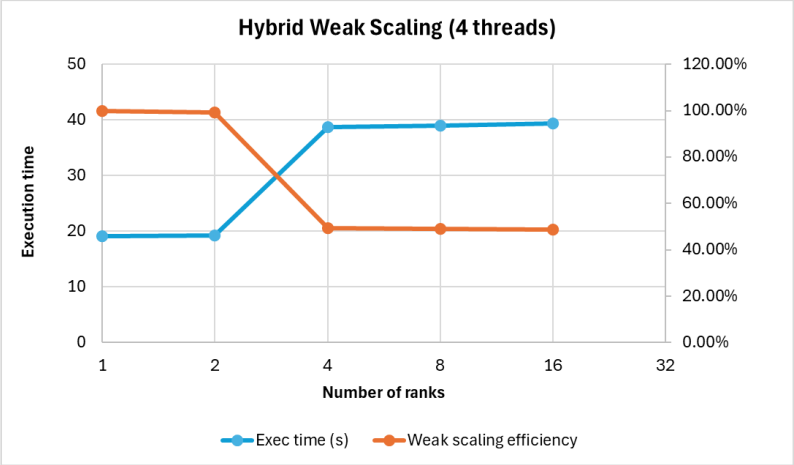


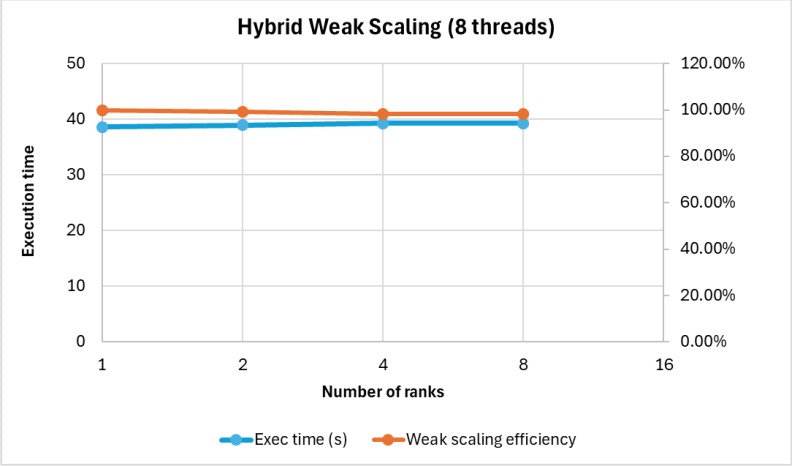
The strong scaling of the Hybrid methods shows two main characteristics:

- For 2 and 4 thread we see the superlinear speedup due to improved cache utilization
- Further increasing the number of processes, we have that the speedup becomes progressively less efficient

Scalability analysis: Hybrid weak scaling



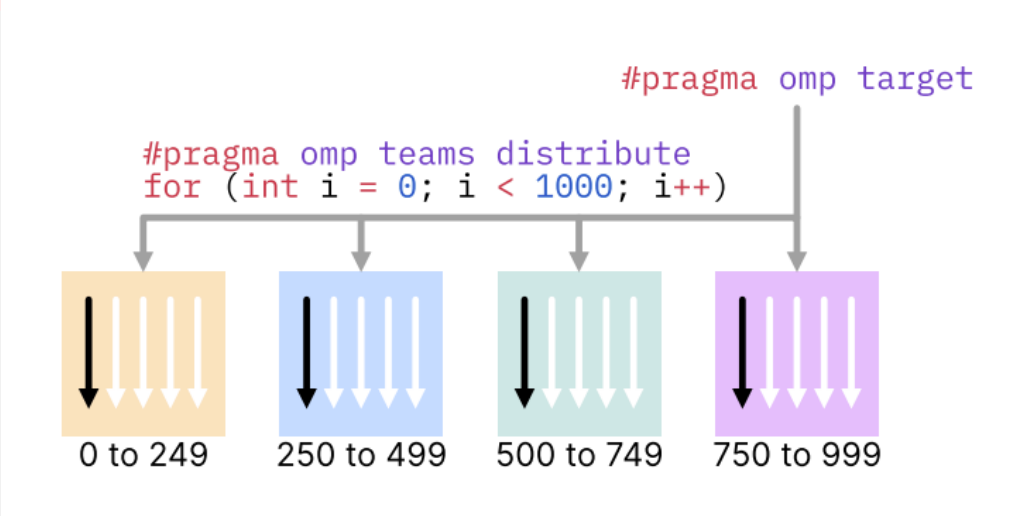




From the weak scaling on the other hand, we can observe that:

- The drop in efficiency that results from increasing the number of ranks is reduced increasing the number of threads
- 40 seconds represents an upper limit to the execution times

GPU parallelization



Lastly, we implemented OpenMP for **GPU** programming

```
#pragma omp target teams distribute parallel for collapse(2)
for(int j = 0; j < ny; ++j) {
  for(int i = 0; i < nx; ++i) {
```

The main bottleneck in such implementation is characterized by the communication to and from the device, This was solved by creating a structured data region, and keeping the data on the device

```
#pragma omp target enter data map(to: eta, h_interp_u, h_interp_v, u, v)
#pragma omp target enter data map(to: h_interp_u.values[0:nx*ny], h_interp_v.values[0:nx*ny])
#pragma omp target enter data map(to: u.values[0:(nx+1)*ny], v.values[0:nx*(ny+1)], eta.values[0:nx*ny])
```

This allowed to reach a **x10 speedup**

Real bathymetry data

A more complex bathymetry data was used to test the implementation.

- Data were taken from *https://emodnet.ec.europa.eu/geoviewer/#*
- **EMODnet** is the European Commission's in situ marine data service that provides free and access to marine environmental and human activities data, supporting research, policymaking, and sustainable ocean management
- The data was selected from a small portion of *Thyrrhenian Sea* containing the *Pianosa* island
- *150x150* grid
- Value of the island not available -> NaN values

Real bathymetry data

The data are given with latitude and longitude coordinates.

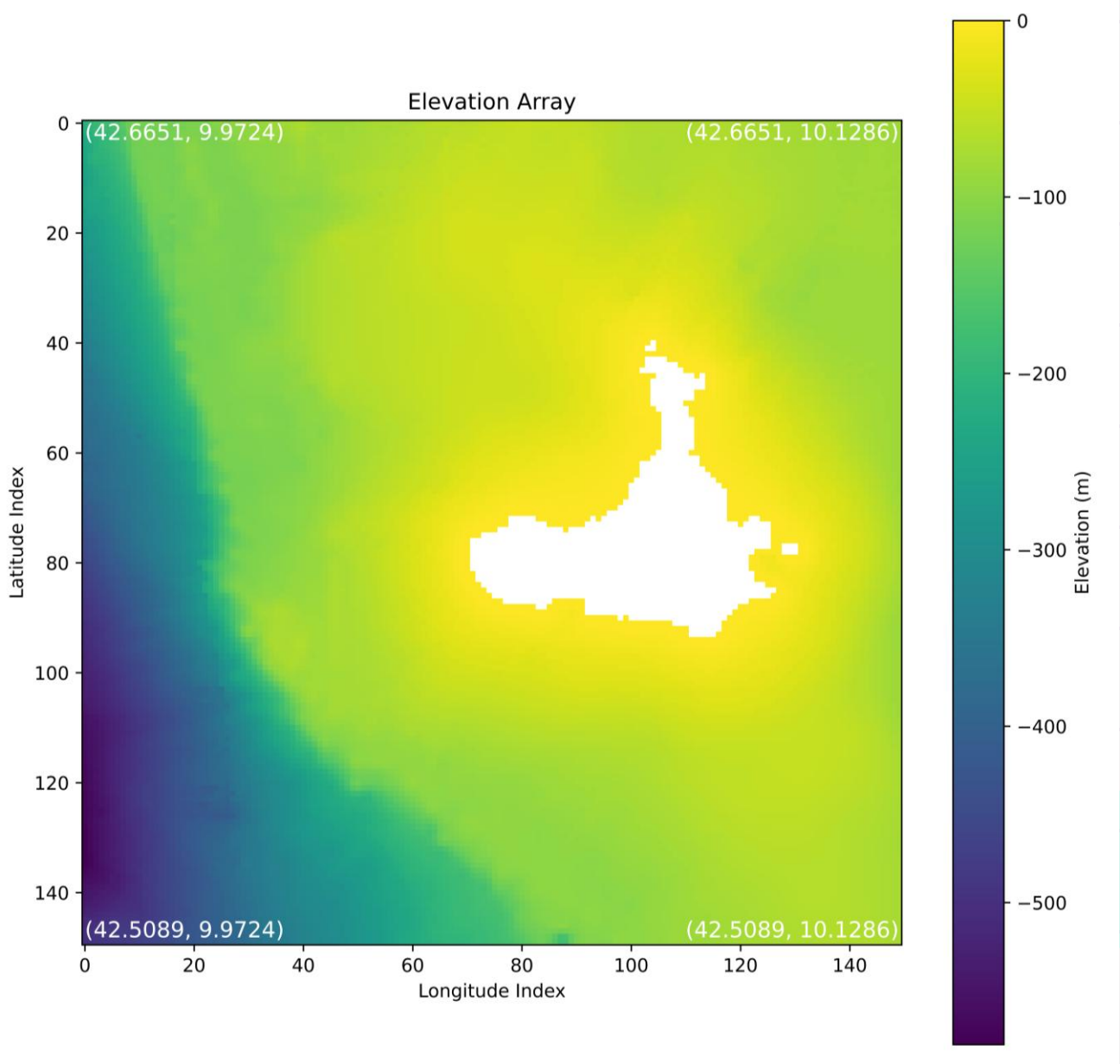
A conversion was necessary to map the values to the data format needed from the code.

In particular a calculation of *dx* and *dy* (discretization step in each direction) was needed.

Assumptions:

- Constant discretization step based on the latitude mean value
- Equal discretization step on the two axis

We created the binary bathymetry file using a Python executable



Real bathymetry data

Calculation steps

- We calculated the absolute value of the difference between the max/min value of each directions (value in degree)
- We divided the value by the number of points (*n=150)*
- We converted the degree to arc seconds
- Assuming the characteristic latitude as the average latitude (*42.58*) we calculated the discretization step in meters (using opendem)

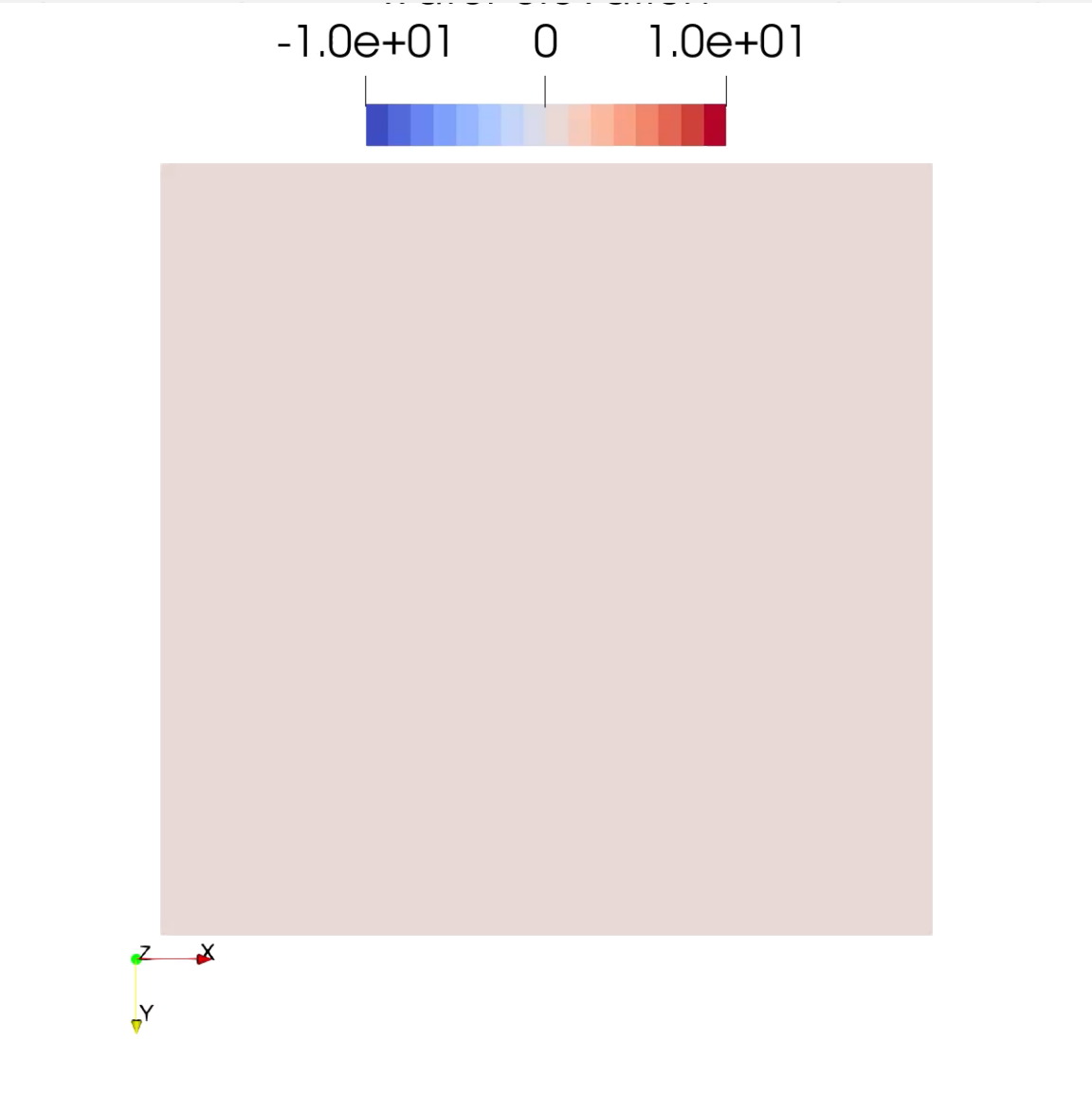
	latitude	**longitude**
absolute difference between max and min (in degree)	0.1562	0.1563
discretization step (in degree)	0.0010417	0.0010417
discretization step (in arc seconds)	3.75012	3.75012
discretization step based on average latitude (in meters)	$85.22 \approx 85$	$85.22 \approx 85$

Real bathymetry data

Three different sources:

1. **Sinusoidal wave on the left side of the dataset**
2. Sinusoidal wave on the upper left corner
3. Sinusoidal wave on the upper left corner and on the lower right corner (with opposite intensities)

The island reflects the waves.

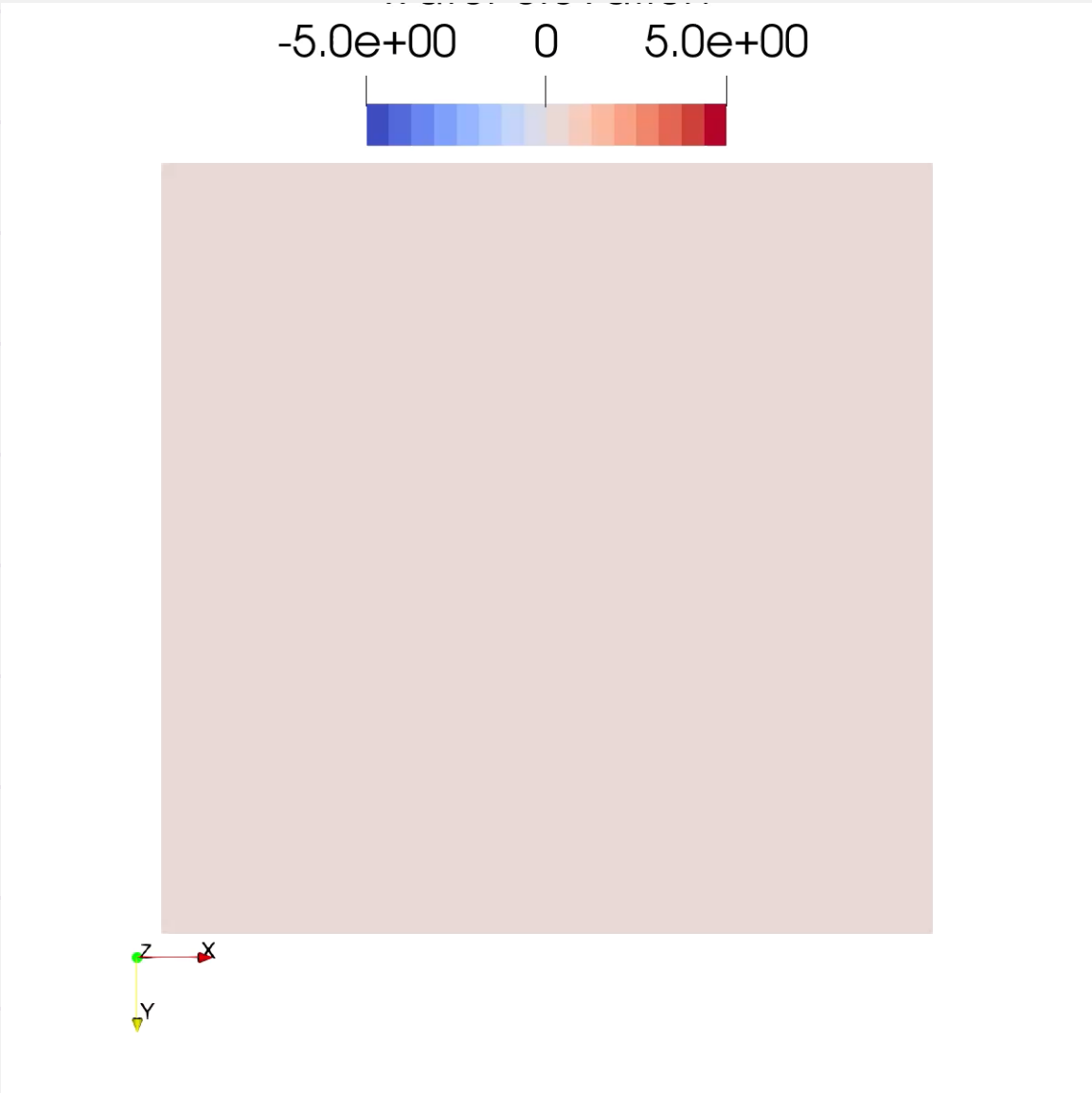


Real bathymetry data

Three different sources:

1. Sinusoidal wave on the left side of the dataset
2. **Sinusoidal wave on the upper left corner**
3. Sinusoidal wave on the upper left corner and on the lower right corner (with opposite intensities)

The island reflects the waves.

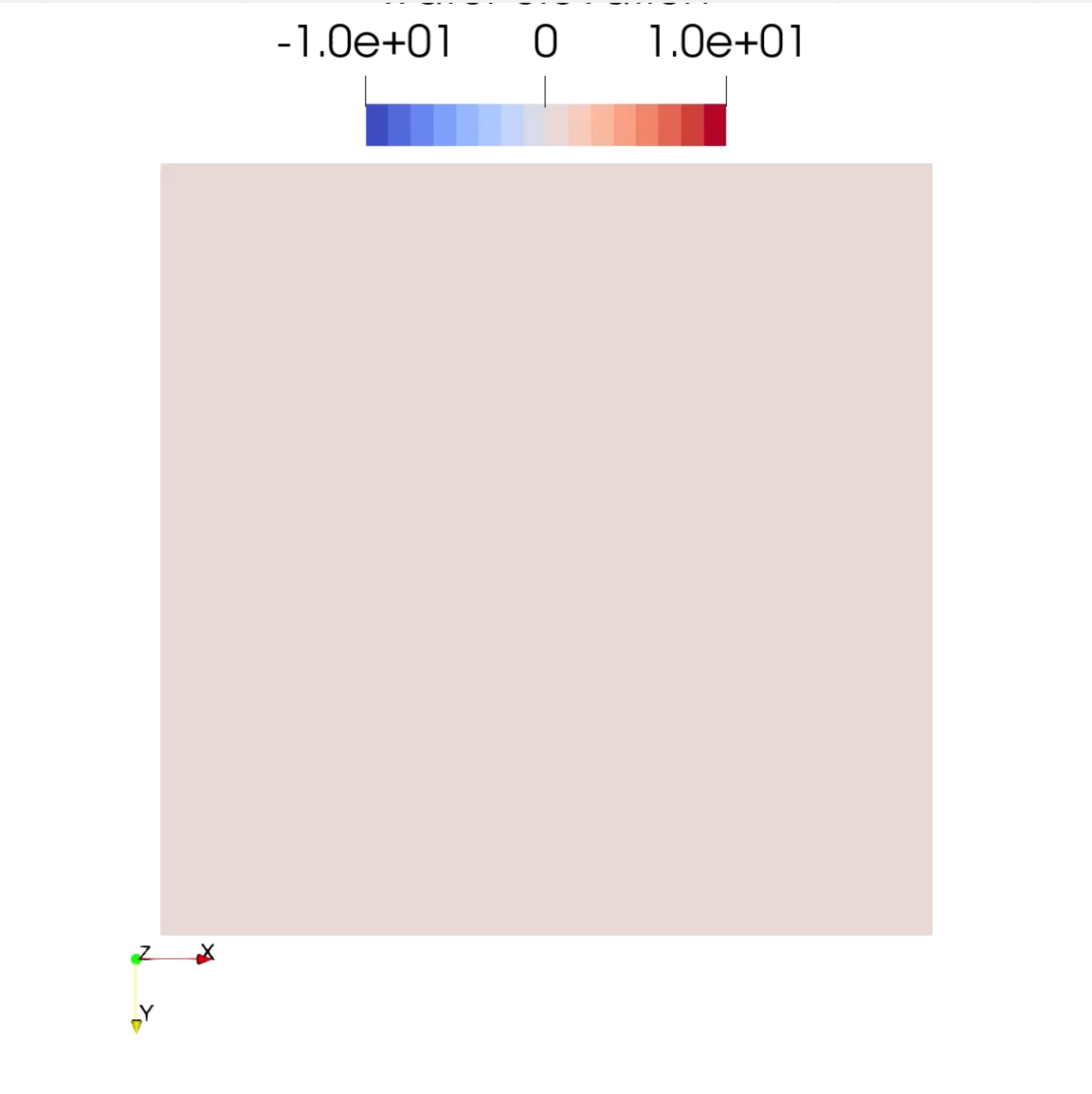


Real bathymetry data

Three different sources:

1. Sinusoidal wave on the left side of the dataset
2. Sinusoidal wave on the upper left corner
3. **Sinusoidal wave on the upper left corner and on the lower right corner (with opposite intensities)**

The island reflects the waves.



Future developments

- Experimenting with Dynamic Loop Scheduling in OpenMP
- Increasing GPU speedup
- Implementing dynamic boundaries on island coast and transparent boundaries to the domain
- Implementing implicit integration scheme to enable more parallelism

LIÈGE université

High Performance Scientific Computing

Thanks for listening, Questions?

Students:
Tommaso Botarelli s2404989
Francesco Moglia s2405535